Week 13: Big Data Processing (Spark)

Keeyong Han

Table Of Contents

1. Lab 1 Feedback & Lab 2 Expectation
2. Recap of NoSQL
3. What is Big Data?
4. Characteristics of Big Data Processing
5. History of Big Data Processing Framework
6. Spark Overview
7. Break
8. Dataframe: R -> Pandas -> Spark
9. Spark Programming
10. Course Update
11. Demo & Homework #9
12. Quiz #3

Lab 1 Feedback

Lab 1 Overall Feedback

- System Diagram must be visual and high level showing data flow and components of your system
- Table Description should cover all tables
- Airflow codes in general
  - Use Airflow features (Connection, Variable)
  - Use Snowflake SQL (MERGE for example)
  - Make sure your code has idempotency (SQL transaction)
  - Two separate DAGs and they need to be scheduled in sequence
- IEEE format (abstract, **conclusion**, **references** and so on)
  - https://www.ieee.org/conferences/publishing/templates.html
- GitHub Readiness (**No deduction this time**)
  - README.md, dags folder, docker-compose.yaml
  - Imagine you put the project repo in your resume

Proper Airflow **Connection** Usage

```
def return_snowflake_conn():

    # Initialize the SnowflakeHook
    hook = SnowflakeHook(
        snowflake_conn_id='snowflake_conn'
    )

    conn = hook.get_conn()
    return conn.cursor()
```

```
def return_snowflake_conn():

    user_id = Variable.get('snowflake_userid')
    password = Variable.get('snowflake_password')
    account = Variable.get('snowflake_account')

    # Establish a connection to Snowflake
    conn = snowflake.connector.connect(
        user=user_id,
        password=password,
        account=account,  # Example: 'xyz12345.us-east-1'
        warehouse='compute_wh',
        database='dev'
    )
    # Create a cursor object
    return conn.cursor()
```

Lab 2 Expectation

- Whatever mentioned in Lab 1 Feedback
  - All of these points are included in the grading criteria for the Group Project
- Using dbt is a must and dbt project needs to be scheduled via Airflow
  - The table generated from dbt should be used in the visualization
- Any dashboard is fine. It doesn’t have to be Superset/Preset
  - Tableau, Power BI, Looker and so on
  - Capture enough screenshots

Recap of NoSQL

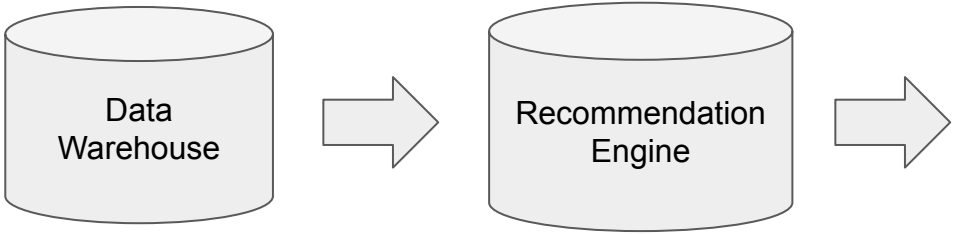
What is NoSQL

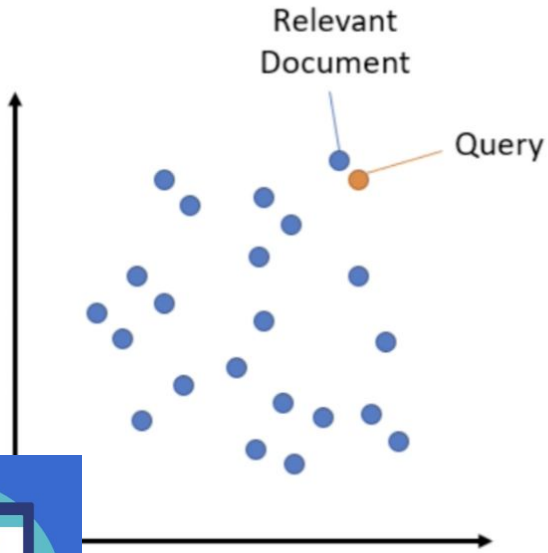
- Non-relational databases
- Schema-less or loose schema
  - Schema can change over time
- Key-Value Storage with different flavors
  - Key/Value Storage, Document Store, Wide Column Storage Search Engine
  - Example: Redis, MongoDB, Cassandra/CouchDB, ElasticSearch/VESPA
- Used in Production
  - Response time is critical (100ms)
  - Needs to be scalable (a few billion records)

Must deliver sub-100ms responses while scaling to billions of records in loose schema

What is Embedding and VectorDB

- Purpose of Embedding
  - Dimensionality Reduction
  - Similarity Measurement (“semantics”)
- VectorDB
  - Convert documents and a query to the same embedding space as vectors
  - Search can be done a similarity search (“Euclidian”, “Cosine”, …)





What is Big Data?

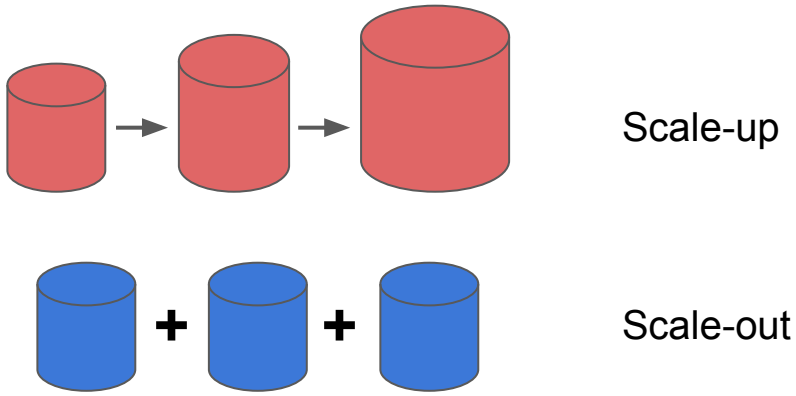
Definition of Big Data #1

- "The size of data that cannot be processed by a single server"
- At the Amazon Cloud Conference in April 2012, Amazon's data scientist, John Rauser, gave this definition, focusing on whether a distributed environment is necessary

What if you can’t process data with Pandas any more?

Definition of Big Data #2

- "Data too large to be handled by traditional software"
- Typical traditional S/W includes relational databases like Oracle or MySQL
  - Not designed with a distributed environment in mind
  - Scale-up approach (vs. Scale-out)
    - Adding more memory, CPU, and disk storage



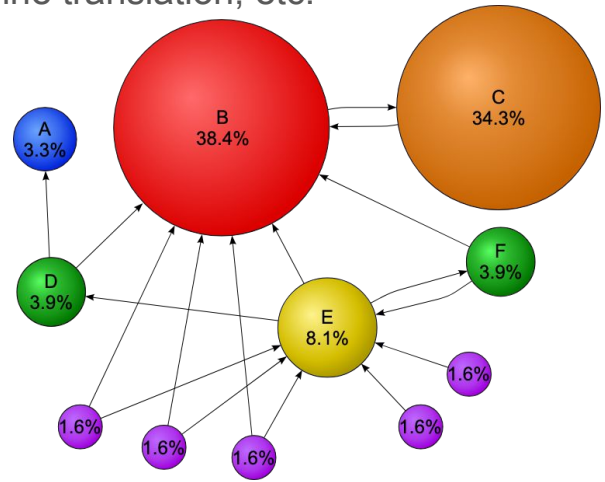
Examples of Big Data: Device Data

- Mobile devices
  - Location data
- Smart TVs
- Various sensor data (IoT sensors)
- Network devices
- …

Examples of Big Data: The Web

- There are tens of trillions of web pages (a vast sea of knowledge)
  - Developing a web search engine is a true large-scale data processing task
    - crawling, ranking (PageRank), indexing them, and serving results
- Google has made significant contributions to the IT industry
  - Even user search queries and click data themselves are large-scale data
  - Analyzing trends based on search queries, statistical machine translation, etc.
- The web itself is training data for developing LLM

Page Rank Algorithm



Characteristics of Big Data Processing

What Makes Big Data Processing Hard?

- Need a way to store large volumes of data without loss: Storage
- Sequential processing takes too long: Computing Power + Parallel Processing
- Big data is often unstructured: SQL alone is not sufficient

- For example, web log files are typical of this kind of data.

```
66.249.65.107 - - [08/Oct/2007:04:54:20 -0400] "GET /support.html HTTP/1.1" 200 11179 "-" "Mozilla/5.0
(compatible; Googlebot/2.1; +http://www.google.com/bot.html)"
111.111.111.111 - - [08/Oct/2007:11:17:55 -0400] "GET / HTTP/1.1" 200 10801
"http://www.google.com/search?q=log+analyzer&ie=utf-8&oe=utf-8
&aq=t&rls=org.mozilla:en-US:official&client=firefox-a" "Mozilla/5.0 (Windows; U; Windows NT 5.2; en-US;
rv:1.8.1.7) Gecko/20070914 Firefox/2.0.0.7"
111.111.111.111 - - [08/Oct/2007:11:17:55 -0400] "GET /style.css HTTP/1.1" 200 3225
"http://www.loganalyzer.net/" "Mozilla/5.0 (Windows; U; Windows NT 5.2; en-US; rv:1.8.1.7) Gecko/20070914
Firefox/2.0.0.7"
```

What are the Solutions?

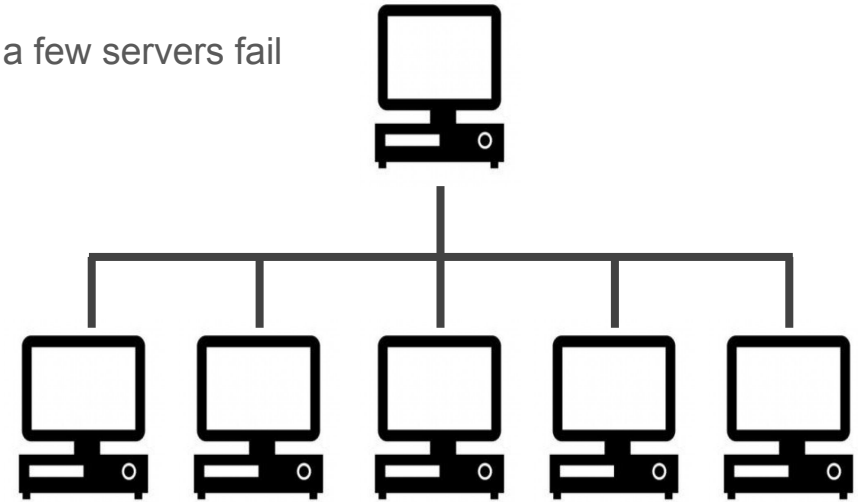
- Need to store large amounts of data without loss.
  - A distributed file system capable of storing large data is required
  - A large file is divided into blocks (128MB) and a block is replicated at least 3 times
- Processing takes a long time.
  - A distributed computing system that can handle parallel processing is necessary
  - A computing job is done by multiple computers (tasks: a task take cares of a data block)
- This data is likely to be unstructured.
  - A method to process unstructured data is required.

In the end, a framework composed
of multiple servers is required.

What is a Large-Scale Distributed System?

- Based on a distributed environment (composed of 1+ servers)
  - Requires a distributed file system and distributed computing system
- Fault Tolerance
  - It must continue to operate even if a few servers fail
- It must be easy to scale
  - Scale Out capability is necessary

Single Server System



Distributed System

History of Big Data Processing Framework

Hadoop: 1st Big Data System

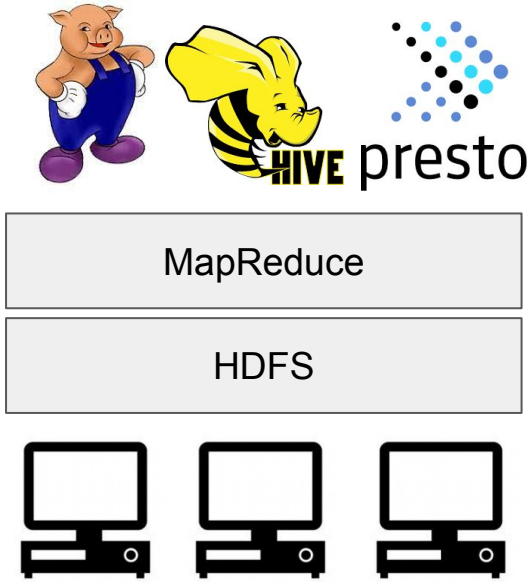
- Doug Cutting developed Hadoop based on research papers by Google Labs
  - 2003: The Google File System
  - 2004: MapReduce: Simplified Data Processing on Large Clusters
- Hadoop started as a sub-project of the open-source search engine, Nutch.
  - The name "Hadoop" comes from the name of Doug Cutting's son's toy elephant.
  - In 2006, it became a separate top-level project under Apache.
- Definition:
  - An open source software platform for **distributed storage** and **distributed processing** of very large data sets on computer clusters built from commodity hardware
  - A cluster system composed of multiple servers (as many as a few tens of thousands)
    - Operates as if it were one giant computer

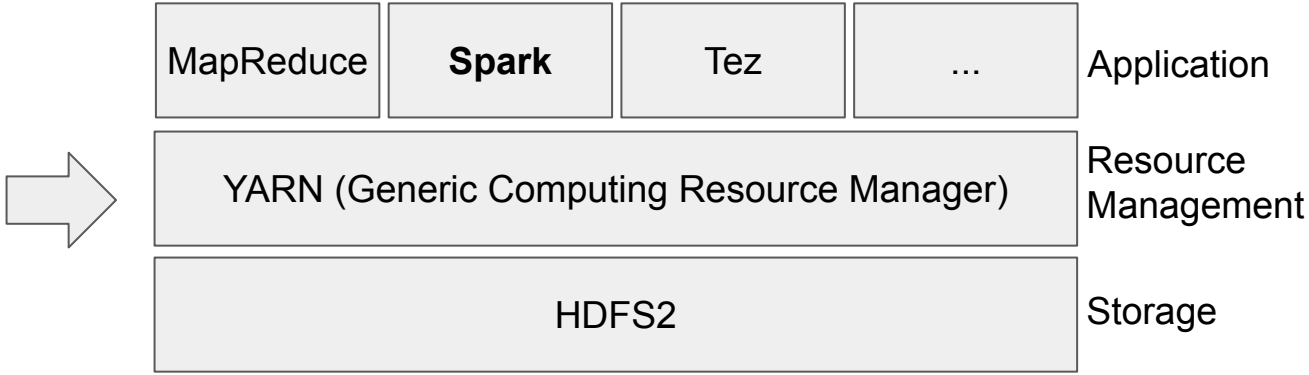
HDFS

MapReduce

Evolution of Hadoop

- Hadoop 1.0 -> 3.0: MapReduce is no longer mainstream



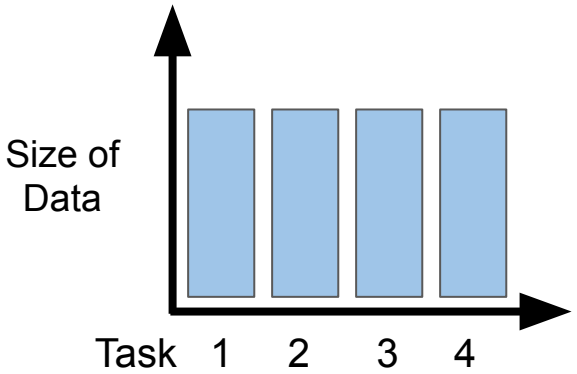


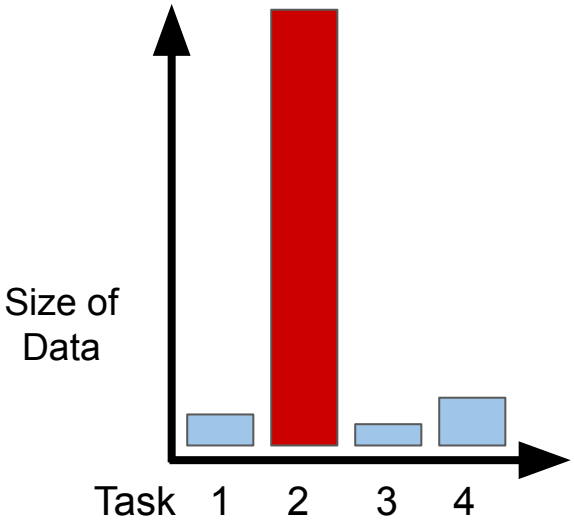
Big Processing Language

- Emergence of MapReduce
  - Only supports two operations - Map and Reduce
  - The only data structure supported is Key, Value(s)
  - Disk based system (High throughput but High latency)
    - Only good for batch processing
  - Productivity is too low
- Comeback of SQL
  - Eventually SQL was reinvented on top of MapReduce: Pig, Hive, Presto, …
- SQL isn’t enough: DataFrame
  - **Spark** was born (2012) from AMPLab in UC Berkeley
    - Now it is a decacorn company - Databricks
  - Again for structured data, SQL is still dominant (Spark SQL)

Data Skew: Nightmare in the Big Data System

- What if there's an imbalance in the size of data each task processes?
  - Parallel processing loses much of its significance
  - The slowest task determines the overall processing speed
  - This applies to operations like Group By or Join
- One of the major issues data engineers struggle
- This problem exists in all big data systems





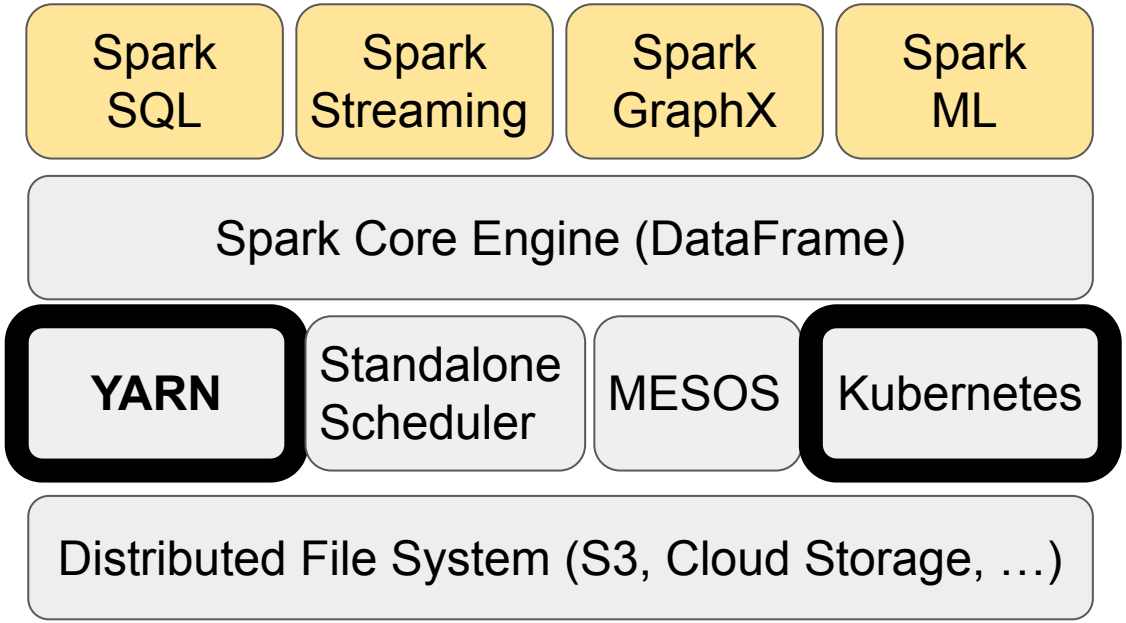
Spark Overview

The emergence of Spark

- Started as an Apache open-source project in 2012 from UC Berkeley's AMPLab
  - Later led to the founding of the startup Databricks
  - Written in Scala (Java)
- Second-generation big data technology following Hadoop
- Provides performance up to 100x faster than MapReduce
  - Disk -> Memory
- Doesn’t have its own computing or file systems
  - Can run on top of YARN or K8s for computing resource management
  - Relies on existing Distributed File Systems (such as HDFS, S3, Cloud Storage, …)
- Provides diverse functionalities for big data processing!
  - DataFrame, SQL, ML, Streaming, Graph, …

Key Components of Spark

- Spark Core
- Spark SQL
- Spark ML
- Spark Streaming
- Spark GraphX



Spark Programming Options

- RDD (Resilient Distributed Dataset)
  - Low-level dataframe - allowing fine-grained control
  - However, it increases coding complexity and not used much at this point
- DataFrame & Dataset (similar to pandas DataFrame)
  - High-level programming APIs, increasingly being used
- Spark SQL
  - Typically used for structured data manipulation

When are DataFrame/Dataset specifically needed?

- For tasks that can't be accomplished with SQL alone

Spark SQL

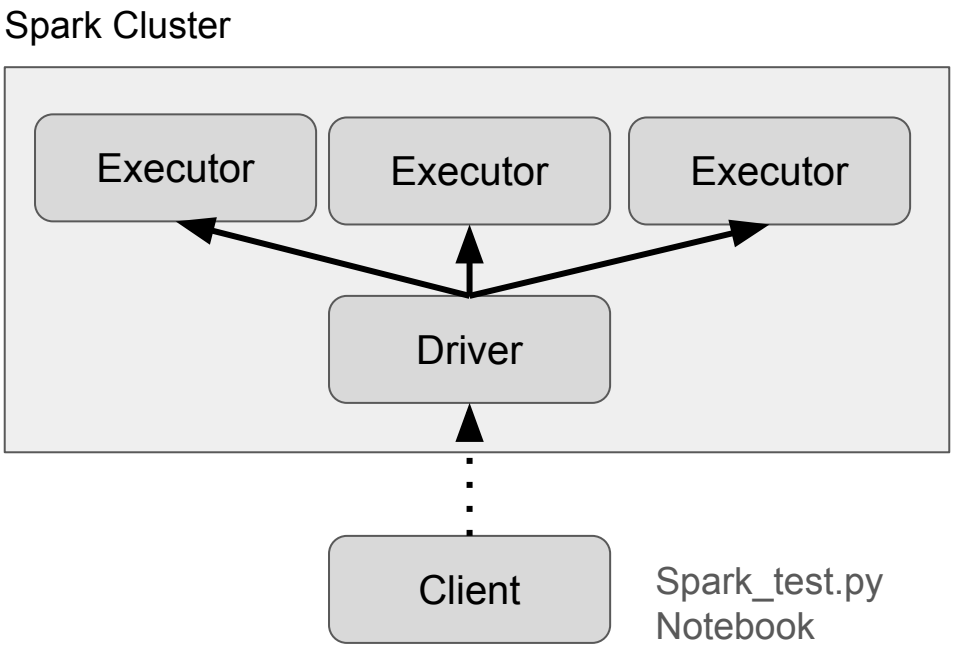
- Spark Data frames can be processed using SQL
  - Data frames can be treated like tables and processed with SQL
- Pandas also provides similar functionality
  - There is a python package called “pandasql”
- We will learn both today

Spark ML

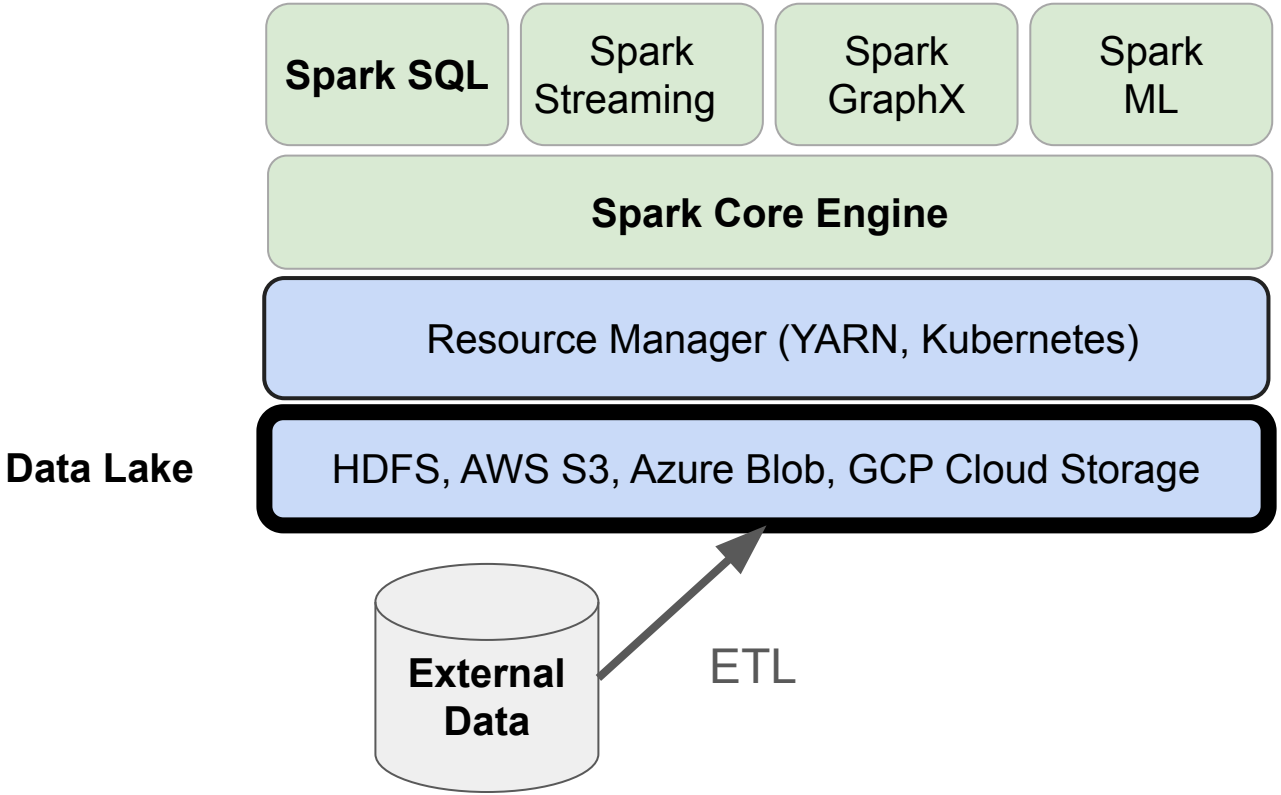
- A library composed of various algorithms and utilities W.R.T machine learning
  - Includes algorithms for Classification, Regression, Clustering, Collaborative Filtering, …
  - For the complete list, refer here. Support for deep learning is limited
  - Supports Distributed training
  - But doesn't support Deep Learning natively
- There are two versions
  - One based on RDD and the other based on DataFrame: spark.mllib vs. spark.ml
- One-stop ML Framework!
  - Data Preprocessing using DataFrames and SparkSQL
  - Model Building with Spark ML
  - Automate model building with ML Pipelines
  - Manage and serve models with **MLflow** (MLOps)

Structured of Spark Program

- Client
  - Original Spark code run by developer
- Driver
  - Acts as the master for the running code submitted by the client
  - Manages executors
- Executor
  - Executes the actual +1 tasks
  - A specific number of CPUs and memory are allocated to each executor. # of CPUs determines how many tasks an executor can run in parallel

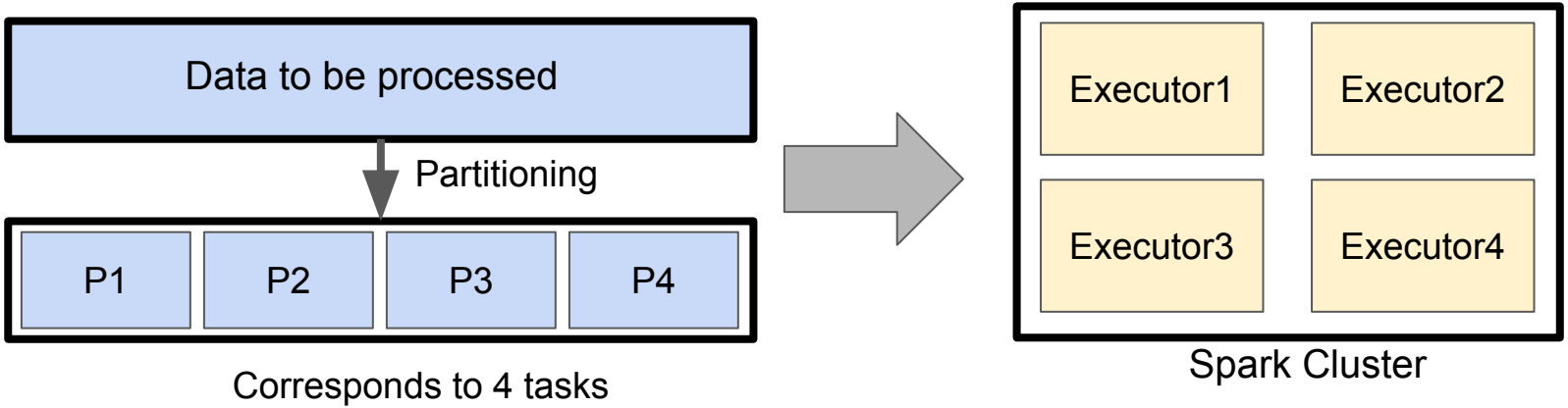


Spark System Architecture (Batch)



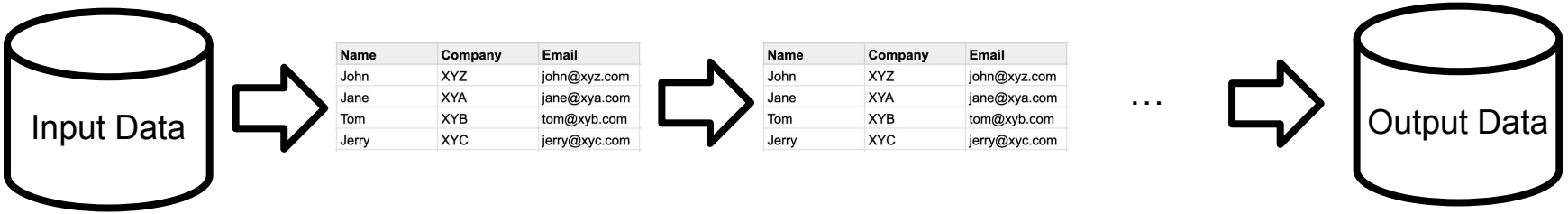
How can Data be Processed in Parallel?

- First, the data must be distributed.
  - Data needs to be partitioned and distributed as blocks/partitions (128 or 256MB)
  - # of partitions in an input data determines the parallelism
- Next, each partition is processed separately and concurrently.
  - data is loaded into memory by partition, and an executor is assigned to handle each partition
  - In terms of programming, we work with DataFrame which consists of partitions



Pandas Data Processing Flow

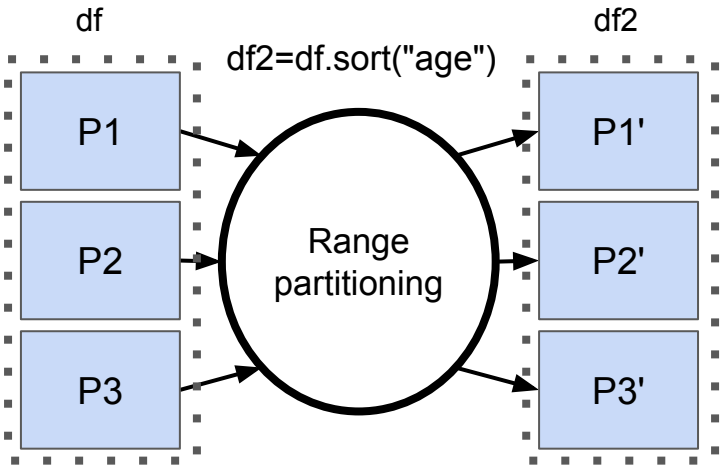
- A DataFrame is composed of small partitions.
  - Once a DataFrame is created, it cannot be modified (Immutable)
  - It is divided into multiple partitions behind the scene and each partition becomes a task
- Continuous DataFrame transformations happen
  - Until the desired DataFrame is created
  - Transformation operations include sort, group by, filter, map, join, etc.



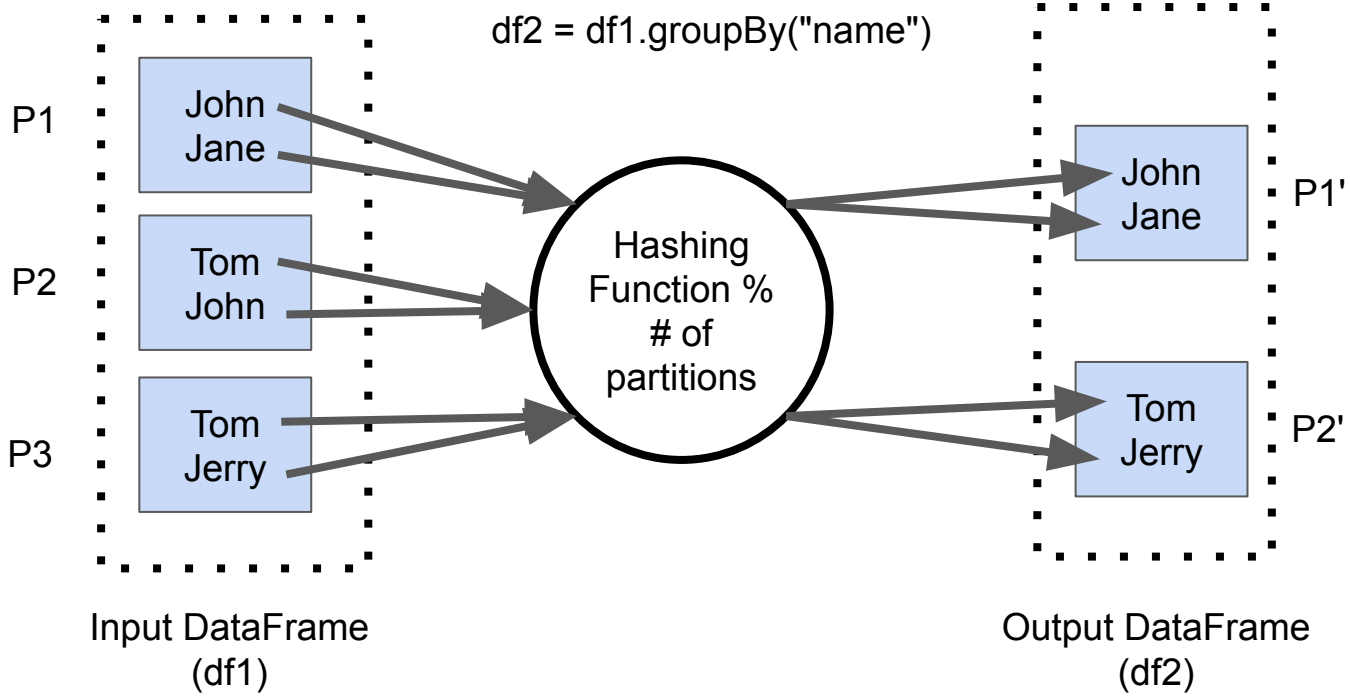
Whenever you do join or group by, new partitions need to be created

Shuffling: when data movement forces new partition creation

- When does shuffling occur?
  - When explicitly creating new partitions
    - e.g., reducing the number of partitions
  - Shuffling initiated by the system
    - Such as aggregations like grouping or sorting.
- Shuffling forces creation of new partitions
  - How many partitions will be created as a result?
    - determined by spark.sql.shuffle.partitions setting
      - The default value is 200
    - The # of partitions is determined by the type of operation performed
      - random, hashing partition, or range partition
- Additionally, **Data Skew** can occur

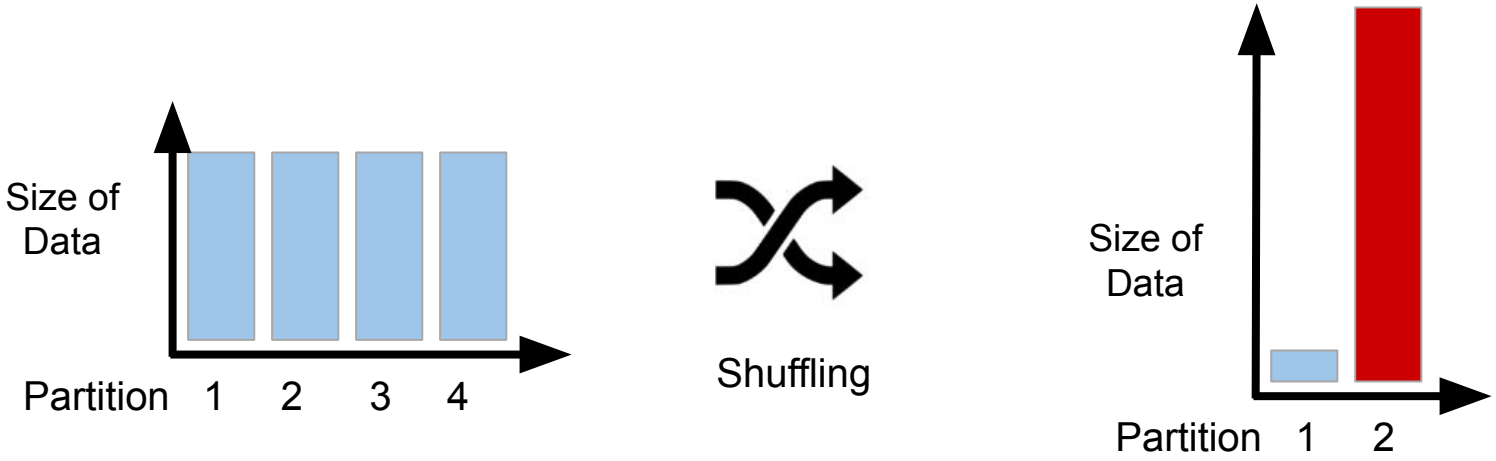


Shuffling: hashing partition (such as groupBY)



Data Skewness: Still a major problem in Spark

- Data partitioning provides parallelism but also has drawbacks
  - This occurs when the data is not evenly distributed
  - It often happens after data shuffling.
- Minimizing shuffling is important, and optimizing partition size is crucial.



Break

Dataframe: R -> Pandas -> Spark

Let’s Process a CSV file in Pandas

- We will do the same processing Spark later
- We will use Google Colab for both Pandas & Spark
- In Pandas we will do the same processing
  a. DataFrame -> pandasql
- CSV file to process
  a. https://s3-geospatial.s3.us-west-2.amazonaws.com/trips.csv
- Find the hours with the most demand in terms of departure time
  - Pandas Code

https://gemini.google/students/

Google Gemini:
Free Pro Plan for Students.

Free for 1 year. Get more access to our most accurate model 3 Pro, unlimited image uploads, customized quizzes and advanced learning tools like NotebookLM, plus 2 TB storage. Just for Students.

Get offer

Terms apply

Spark Programming

Spark Programming Environment

- We will use PySpark 3.5.x
- We will use Google Colab

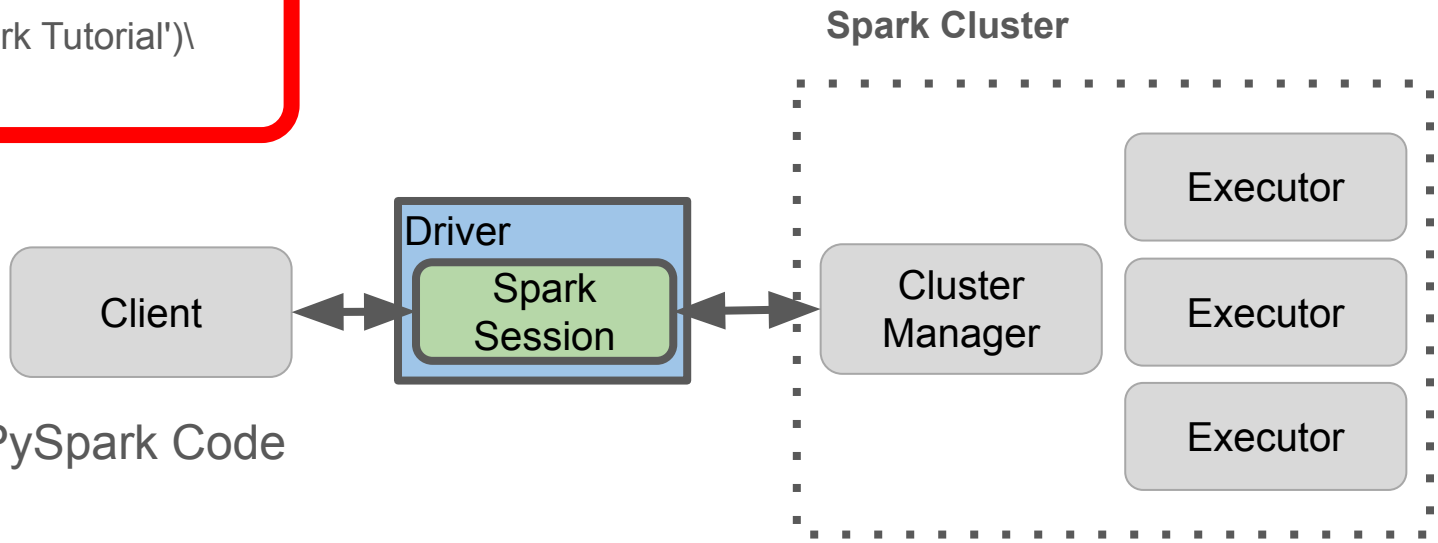
How to Begin Programming: Creation of Spark Session

- First, create a SparkSession object to communicate with the Spark Cluster
  - With a SparkSession, you can work with DataFrames, SQL, Streaming, and the ML API.
- You can configure various settings using the config method.
- Refer to the Spark Session API documentation for more details.

SparkSession Creation Example in PySpark

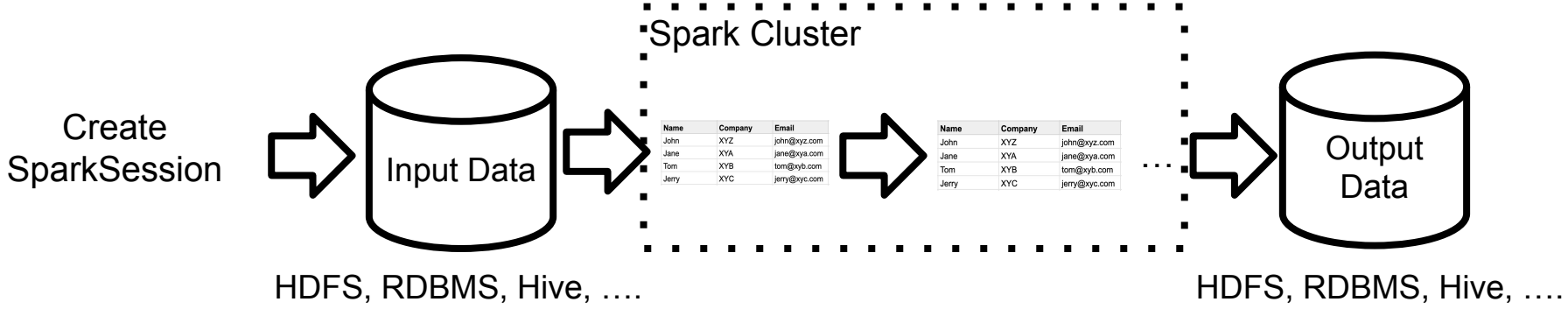
```
from pyspark.sql import SparkSession

# SparkSession is a singleton
spark = SparkSession.builder\
  .master("local[*]")\
  .appName('PySpark Tutorial')\
  .getOrCreate()

…

spark.stop()
```

In Pandas, everything happens in a server
In Spark, a separate set of servers is used in production environment



Overall Code Flow

1. Creating a SparkSession
2. Loading the input data as a DataFrame
3. Performing data manipulation tasks (very similar to Pandas)
   a. Using the DataFrame API or Spark SQL
4. Continuously creating new DataFrames until the desired result is achieved
5. Saving the final result



Porting what we built in Pandas to Spark

- Concepts of DataFrames and dominant use of SQL for structured data processing are the same between Spark and Pandas
- Spark Code

Course Update

Week 14 to Week 17

- Week 14 (11/25)
  - Streaming Data Overview
  - **Lab 2 Due**
  - Guest Speaker (Zoox)
- Week 15 (12/2)
  - End-to-End Data System: Revisiting what we learned so far
  - Guest Speaker (Apple)
- Week 16 (12/9)
  - Group Project Presentation (8 mins presentation, 2 min Q&A)
    - Presentation order will be announced on 12/2
- Week 17
  - **Final Exam**

Course Update: Quizzes, Group Project, and Final Exam

- Quizzes
  - 2 best out of 3 will be used for grading after normalization
- Group Project
  - Make sure scope of your project isn't too big
  - Make sure you are a team player
- Final Exam
  - Covers from dbt to the end
  - 3 questions on writing SQLs
    - Study Easy "Advanced Select and Joins" & "Basic Aggregate Functions" at https://leetcode.com/studyplan/top-sql-50/

Demo & Homework #9

Companies Using Apache Spark

Powering data processing at scale across the world's biggest tech companies.

PySpark vs Spark SQL

Aspect	PySpark (DataFrame API)	Spark SQL
Interface Type	Pythonic, object-oriented	SQL query-based
Who Uses It	Data engineers, developers	Data analysts, SQL users
Code Style	Uses Python methods like `select()`, `filter()`, `groupBy()`	Uses standard SQL syntax (`SELECT ... FROM ...`)
Flexibility	Great for dynamic logic, loops, and conditional transformations	Great for declarative queries and readability
Performance	Identical — both compile to the same execution plan	Identical — both use Catalyst optimizer
When It Shines	When integrating with Python libraries (e.g., ML, Pandas UDFs)	When writing ad-hoc or analytical queries

Demo: Colab Link

- Read log files and save the result as a table to Snowflake

df

123.45.67.89 - - [05/Nov/2024:02:08:16 +0000] "DELETE /cart HTTP/1.1" 500 242 192.168.1.1 - - [04/Nov/2024:21:23:39 +0000] "POST /checkout HTTP/1.1" 404 2781
234.56.78.90 - - [05/Nov/2024:07:06:19 +0000] "GET /api/data HTTP/1.1" 301 3758 192.168.1.1 - - [04/Nov/2024:20:03:56 +0000] "POST /home HTTP/1.1" 200 1837 192.168.1.1 - - [04/Nov/2024:21:25:05 +0000] "GET /products/123 HTTP/1.1" 200 3430

df_log

ip	timestamp	method	url	status	size
123.45.67.89	05/Nov/2024:02:08...	DELETE	/cart	500	242
192.168.1.1	04/Nov/2024:21:23...	POST	/checkout	404	2781
234.56.78.90	05/Nov/2024:07:06...	GET	/api/data	301	3758
192.168.1.1	04/Nov/2024:20:03...	POST	/home	200	1837

url_404_count

url	count
/products/123	300653
/home	300048
/api/data	299683
/cart	299543
/checkout	299506

url_404_count table

Homework #9 (9 points): The last homework

Modify today's Demo Colab content and create a table with ip and method count in Snowflake (refer to the image in the slide)

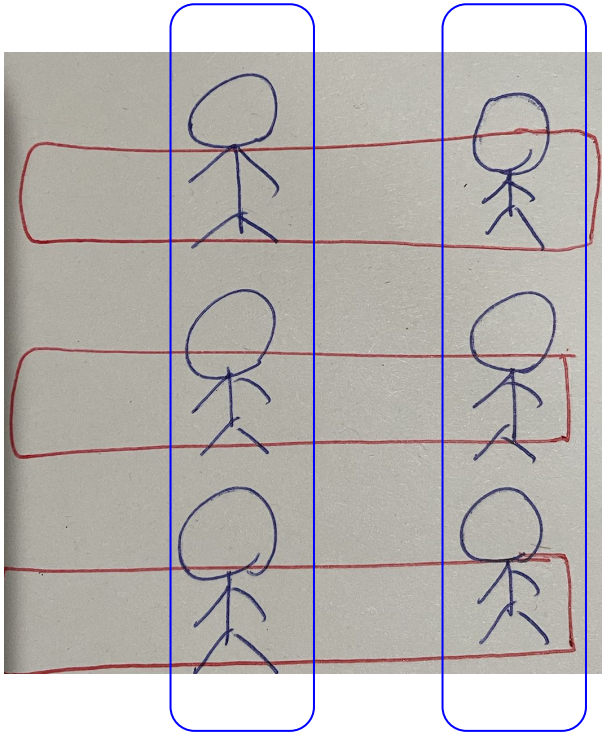
1. Download the 7 log files using the links provided in the demo Colab notebook (+1 pt)
2. Configure Spark environment as shown in the demo notebook (+1 pt)
   a. Add the Snowflake JDBC .jar file and Initialize SparkSession
   b. Create the input DataFrame (df) and parsed DataFrame (log_df)
3. Compute **IP and method** combination counts using DataFrame operations (+2pt):
   a. Count the occurrences of each unique (ip, method) pair
   b. Sort the result in **descending** order by count
4. Repeat step 4 using Spark SQL instead of DataFrame operations (+2 pt)
5. Write the resulting DataFrame from step 4 to a Snowflake table (+2 pt)
6. Capture a screenshot of the Snowflake table from Snowflake Web UI (+1 pt)

```
+------------+------+------+
|          ip|method| count|
+------------+------+------+
|234.56.78.90|   GET|501202|
|123.45.67.89|  POST|500759|
| 192.168.1.1|   GET|500490|
```

For each step, capture screenshots accordingly
Submit your **Google Colab link** in the report PDF

Quiz #3

15 points for 15 minutes

Ground Rules

- 15 questions in 15 minutes
- No bathroom breaks allowed
- Put your digital devices in your own bags.
- Keep your bags on the floor
- Seating Criteria
  - Aligned in the same vertical columns.
  - Seats must be alternated within each row, with no students sitting directly beside each other.